

230 0038 ARSAM ALI TASKS:

Task 1: introduce error intentionally

- Error #1:

```
1      [org 0x0100]
2
3      mov [num1],[num2];
4      mov bx,[num2];
5      add ax,bx;
6      mov bx,[num3];
7      add ax,bx;
8      mov [num4],ax;
9      mov ax,0x4c00;
10     int 0x21;
11
12     num1:dw 5
13     num2:dw 10
14     num3:dw 14
15     num4:dw 0
16
```

(error highlighted in yellow)

```
task1.asm
X:\>nasm task1.asm -o task1.com
task1.asm:3: error: operation size not specified
```

The above command is illegal as you cannot put a address in the address because you cannot move the data to another location without registers.

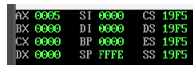
Task #2: Explain example c02-01.asm line by line.

```
1  [org 0x0100]
2
3  mov ax, [num1]      ; load first number in ax
4  ; mov [num1], [num2] ; illegal
5  mov bx, [num2]
6  add ax, bx
7  mov bx, [num3]
8  add ax, bx
9  mov [num4], ax
10 mov ax, 0x4c00
11 int 0x21
12
13
14 num1: dw 5
15 num2: dw 10
16 num3: dw 15
17 num4: dw 0
```

Line 1: This is the starting address of the code

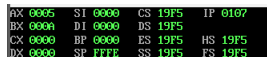
 (starting address is highlighted)

Line 3: value 5(in hex) is moved into ax register

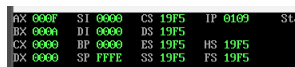
 (after execution of second line ax register stores value 5)

Line 4: line 4 will generate error so its commented (explained in question 1)

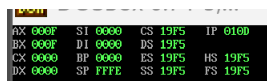
Line 5: Now bx registers stores value 10(as hex “A”) as bx register now points to num2.



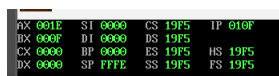
Line 6: adds the value stored in ax and bx register and stores it in ax register.

 (5+A=F which is now stored in AX)

Line 7: now bx points to value 15 and thus bx register stores 15 in hexadecimal (F)

 (bx is changed from “A” to “F”)

Line 8: now as from previous steps both ax and bx store value 15 this 15+15 will be added and 30(in hex) will be stored in ax.

 (30 stored in hex form in AX register)

Line 9: the result (30 in hex) is now moved into num4.

Question #3: For the examples c02-02.asm to c02-06.asm, just explain their difference from example c02-01.asm.

C02-02.asm-c02-06.asm	C01-01
Uses Base + indexed addressing mode	Uses direct addressing mode
<code>mov [num1 + 6], ax</code>	<code>mov ax, [num1]</code>

Question #4:

```

11      mov ax, 0           ; reset the accumulator
12      mov cx, 3           ; set the iterator count
13      mov bx, num1        ; set the base
14
15      outerloop:
16          add ax, [bx]
17          add bx, 2
18
19          sub cx, 1
20          jnz outerloop

```

line 11-13: AX,CX and BX registers are initialized and after initialized they store values as shown below:

```

AX 0000 SI 0000 CS 19F5 IP 0109 Stack
BX 011D DI 0000 DS 19F5
CX 0003 BP 0000 ES 19F5 HS 19F5
DX 0000 SP FFFE SS 19F5 FS 19F5

```

LOOP PART(LINE 15-20):

Line 16 will store value 5 in ax as bx points to num1 :

```

AX 0005 SI 0000 CS 19F5
BX 011D DI 0000 DS 19F5
CX 0003 BP 0000 ES 19F5 HS 19F5
DX 0000 SP FFFE SS 19F5 FS 19F5

```

Line 17 will point to next value in dw :

```

; ...
num1: dw 5, 10, 15

```

Line 19 will subtract cx register by 1 and make it 2 (3-1):

```

AX 0005 SI 0000 CS 19F5 IP 0113
BX 011F DI 0000 DS 19F5
CX 0002 BP 0000 ES 19F5 HS 19F5
DX 0000 SP FFFE SS 19F5 FS 19F5

```

Line 20 : will check the condition (jump if no zero in cx register) by checking if zero flag is raised or not.

OF DF IF SF ZF AF PF CF
0 0 1 0 0 0 0 0 as ZF is zero this loop will iterate again but,

AX	001E	SI	0000	CS	19F5	IP	0113	Stack	+0 0000	Flags	7244
BX	0123	DI	0000	DS	19F5				+2 20CD		
CX	0000	BP	0000	ES	19F5	HS	19F5		+4 9FFF	OF DF IF SF ZF AF PF CF	
DX	0000	SP	FFFF	SS	19F5	FS	19F5		+6 EA00	0 0 1 0 1 0 1 0	

CMD >											
010F	81F90100	SUB	CX,0001								
0113	75F4	JNZ	0109								

ES:0000	CD	20	FF	9F	00	EA	FF	FF
DS:0000	AD	DE	1B	05	C5	06	00	00
DS:0010	10	01	10	01	10	01	32	01

As ZF is raised this loop will break and later program will be terminated.

In conclusion loop iterates until zero flag is 0 as soon it becomes 1 loop will break.